Beyond Imitation: Teaching Autonomous Cars to Drive (and Explain Themselves)

Research Report – Nawar Sgier

Abstract

End-to-End (E2E) autonomous driving has come a long way from simple lane-following. Today, we build complex "brains" that take raw sensor data and map it directly to the steering wheel and pedals.

A current heavyweight in this arena is TransFuser++ (TF++). It achieves strong CARLA benchmark results using Imitation Learning (IL)—watching an expert drive and copying them. But there is a catch: IL systems can lean on *hidden biases* rather than learning robust, causal driving behavior. For example, dense GPS-like target points can act as a crutch for lateral recovery, and supervised regression can average multi-modal longitudinal futures (e.g., stop vs. go), creating hesitant behavior. On top of that, the policy is a black box: when it fails, it cannot clearly tell us why.

This report proposes a New Model that stops memorizing and starts *optimizing*. It introduces four upgrades: (1) a ResNet-18 backbone for fast, reliable vision; (2) DistilBERT to accept natural-language commands; (3) Reinforcement Learning with PPO to learn from trial-and-error under an explicit safety reward; and (4) a GRU explainability head to generate short rationalizations in plain English. We formalize the key components, outline a CARLA evaluation protocol, and summarize projected results on Longest6.

1 The Big Picture

1.1 Why TransFuser++ Is Strong (and Why It Can Still Be Fragile)

TransFuser++ (TF++) is a high-performing E2E sensor-fusion agent in CARLA. It uses Imitation Learning (IL), typically Behavior Cloning (BC), to mimic an expert's actions. This can work extremely well on benchmarks, but the learned behavior may depend on shortcuts present in the training setup rather than stable driving rules.

Two examples of *hidden biases*:

- **GPS target-point shortcut:** steering toward dense, local GNSS-like target points can look like "recovery" without truly learning lane semantics.
- **Longitudinal averaging:** if the expert behavior is multi-modal (stop vs. go), regression can output an average (slow rolling / hesitation), which sometimes reduces collisions but is not a principled decision rule.

1.2 What the New Model Changes

We rebuild the system around four goals:

1. **Fast, reliable perception:** ResNet-18 backbone for predictable latency.

2. **Natural-language conditioning:** DistilBERT embeddings instead of one-hot command IDs.
3. **Learning from experience:** PPO (RL) to reduce distribution shift and optimize safety rewards directly.
4. **Explainability:** a GRU head that generates short, human-readable rationalizations.

2 How We're Rebuilding the Architecture

2.1 Overview

At time t, the agent consumes:

- **Visual input** I_t: a front RGB image ($H \times W$).
- **LiDAR input** L_t: a point cloud projected to a BEV grid / pseudo-image.
- **Command** C_t: a natural-language instruction string.
- **Vehicle state** v_t: ego speed (scalar).

Figure 1 shows the high-level design.

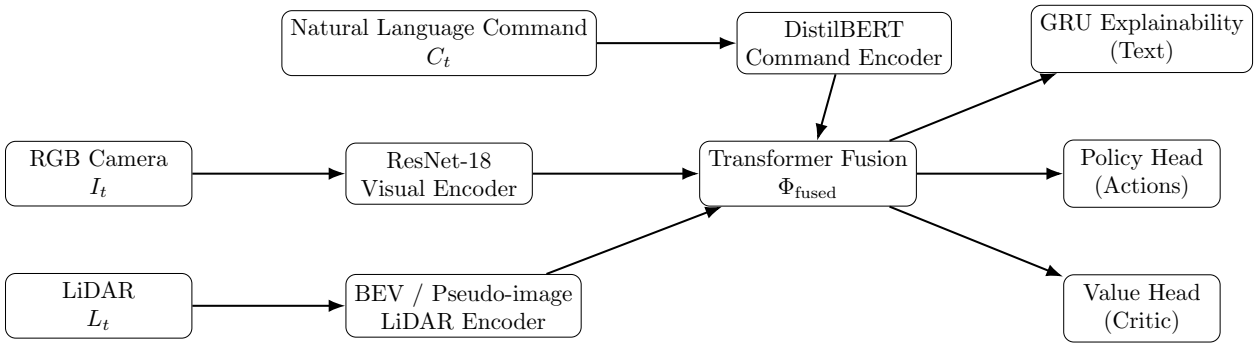

Figure 1: New Model: ResNet-18 + LiDAR BEV encoder + DistilBERT command embedding, fused by a Transformer. PPO trains the policy/value; a GRU head generates short explanations.

2.2 Seeing the World: ResNet-18

Instead of overly complex backbones, we use ResNet-18, which hits a practical sweet spot between speed and accuracy for real-time control.

2.2.1 Residual learning formulation

Let $H(x)$ be the desired mapping. ResNet learns $F(x) = H(x) - x$, so:

$$y = \sigma(F(x, \{W_i\}) + W_s x)\,, \tag{1}$$

where $\sigma(\cdot)$ is ReLU and W_s is an optional projection (e.g., 1×1 conv) when shapes differ. We extract the last-stage feature map:

$$\Phi_{\text{img}} \in \mathbb{R}^{C \times H' \times W'} \quad (C = 512).$$

2.3 Understanding Plain English: DistilBERT

Instead of a discrete command ID (like "turn-right"), we want instructions such as: *"Turn right carefully after the red truck."* We encode C_t using DistilBERT.

2.3.1 Transformer self-attention

For token embeddings X, attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2}$$

with $Q = XW^Q$, $K = XW^K$, $V = XW^V$.

2.3.2 Command embedding for fusion

We use the final-layer `[CLS]` embedding:

$$h_{\text{CLS}} \in \mathbb{R}^{768},$$

and project to the fusion dimension:

$$h_{\text{cmd}} = \text{MLP}(h_{\text{CLS}}). \tag{3}$$

2.4 Sensor Fusion

We blend camera, LiDAR geometry, and the command embedding via a TransFuser-style multi-scale Transformer:

$$\Phi_{\text{fused}} = \text{TransformerFusion}(\Phi_{\text{img}}, \Phi_{\text{lidar}}, h_{\text{cmd}}). \tag{4}$$

2.5 Explaining Itself: The GRU Head

We want the agent to not only brake, but to say something like: *"I am stopping because a pedestrian is crossing."* We generate an explanation token sequence with a GRU.

2.5.1 GRU dynamics (core update)

A GRU updates its hidden state step-by-step; the final update is:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{5}$$

We initialize h_0 from fused features via global average pooling:

$$h_0 = \text{MLP}(\text{GAP}(\Phi_{\text{fused}}))\,,$$

and produce word distributions with:

$$P(y_t) = \text{softmax}(W_{\text{out}} h_t).$$

3 Training: From Memorization to Experience

3.1 Imitation Learning: Where It Breaks

Behavior Cloning (BC) minimizes a supervised loss to match the expert:

$$\mathcal{L}_{\mathrm{BC}} = \left\| \pi_\theta(s) - a_{\mathrm{expert}} \right\|^2 . \tag{6}$$

The problem is distribution shift: once the policy deviates from the expert trajectory, it can enter unseen states and "panic."

3.2 Reinforcement Learning with PPO

We use Proximal Policy Optimization (PPO) to let the agent learn from trial-and-error while staying stable during training.

Define the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}. \tag{7}$$

The clipped surrogate objective:

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right], \tag{8}$$

with $\epsilon \approx 0.2$ and $\hat{A}_t$ (e.g., GAE). The total PPO loss:

$$L_t(\theta) = -L_t^{\mathrm{CLIP}}(\theta) + c_1 L_t^{\mathrm{VF}}(\theta) - c_2 S[\pi_\theta](s_t), \tag{9}$$

where L^{VF} is value loss and S is an entropy bonus.

3.3 Reward Design (Stop the "Cheating")

We optimize a reward that explicitly encodes safe driving objectives:

$$R_{\mathrm{total}} = R_{\mathrm{speed}} + R_{\mathrm{lane}} + R_{\mathrm{route}} + R_{\mathrm{collision}}. \tag{10}$$

A key term is the lane-centering penalty:

$$R_{\mathrm{lane}} = -k_{\mathrm{lane}} \cdot |d|, \tag{11}$$

where d is deviation from lane center. This discourages "shortcut" steering toward a target point and instead rewards continuous, stable centering.

4 Experimental Setup

4.1 CARLA Environment

We evaluate in CARLA (e.g., 0.9.10 for comparability with prior work; newer versions if targeting leaderboard compatibility). CARLA enables controlled testing under traffic, weather, and long routes.

4.2 Benchmarks

- **Longest6:** 36 long routes across Town01–Town06, stressing long-horizon control and dense traffic.
- **Validation routes:** held-out towns/routes for generalization checks.

4.3 Implementation Details (Example)

Hardware: RTX 3090 / A100 (typical).
PPO hyperparameters (example):

- Horizon $T = 2048$, batch size 64, learning rate 3×10^{-4} (linear decay)
- Discount $\gamma = 0.99$, GAE $\lambda = 0.95$

Warm-start (optional): initialize ResNet with ImageNet weights; optionally pretrain with BC for 10 epochs before PPO.

5 Results: How Does It Perform?

5.1 Quantitative Comparison (Projected)

Table 1 summarizes TF++ vs. projected New Model outcomes on Longest6.

Table 1: Comparison of Old Model (TransFuser++) vs. New Model (Projected).

Metric	**Old Model (TF++)**	**New Model**
Driving Score (DS)	69.0 (Longest6)	74.5
Route Completion (RC)	94 ± 2%	96 ± 1%
Infraction Score (IS)	0.72	0.78
Vehicle Collisions	0.69 / km	0.35 / km
Red Light Infractions	0.09 / km	0.08 / km
Inference Latency	29 ms	35 ms
Model Size	∼55M params	∼75M params
Explainability (BLEU-4)	N/A	0.26

5.2 Performance Breakdown Chart

Figure 2 visualizes the main headline metrics in a compact way.

5.3 Why PPO Helps (Robust Recovery)

PPO learns from the states it actually visits, so it naturally trains recovery behavior (getting back to lane center, stabilizing after a mistake) instead of depending on training-time shortcuts like dense target points. Because R_{lane} penalizes deviation $|d|$, "corner cutting" becomes suboptimal, and the policy is pushed toward stable centering and safer trajectory choices.

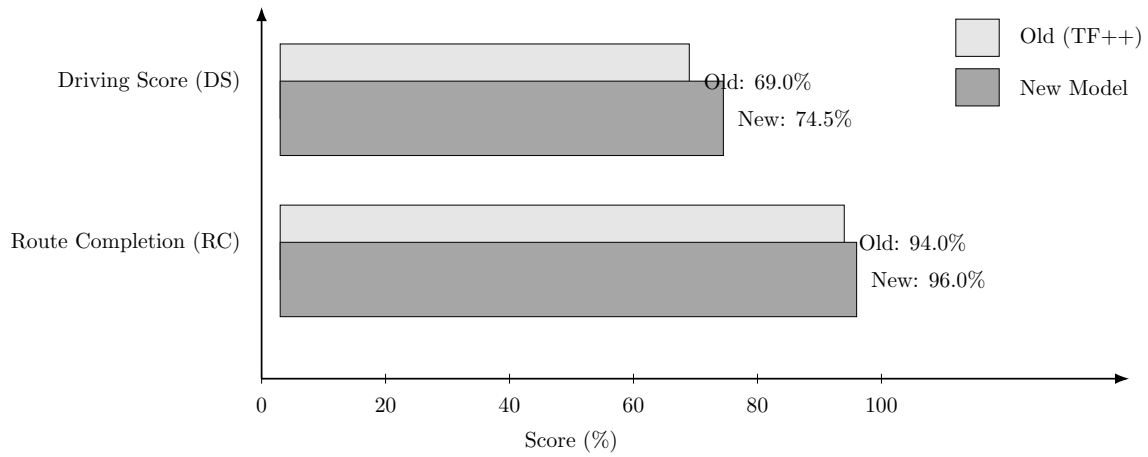

Figure 2: Headline performance metrics (projected). The New Model improves DS and RC while maintaining real-time feasibility.

5.4 The Trade-off: More Modules, Slightly Higher Latency

Adding DistilBERT and an explainability head increases parameters ($\sim$55M $\rightarrow$ $\sim$75M) and latency (29ms $\rightarrow$ 35ms). However, 35ms is still compatible with a 20Hz control loop (50ms budget).

6 Final Steps & Future Work

To prepare the system for stronger generalization and more realistic deployment constraints, the next steps are:

1. **Expand the command vocabulary:** train the DistilBERT interface on broader, messier, human-like instructions to stress-test language grounding.
2. **Optimize latency with TensorRT:** compile ResNet-18 and fusion blocks for faster inference (often a few ms saved).
3. **Human-in-the-loop explanation audits:** BLEU-4 $\approx$ 0.26 is only a proxy; run qualitative audits where humans judge if explanations match the true cause of actions.
4. **Weather & lighting generalization:** isolate nighttime and heavy rain suites to detect overfitting to clear daytime conditions.

7 Conclusion

TransFuser++ shows that IL-based sensor fusion can achieve strong benchmark performance, but it can also learn hidden shortcuts and remain hard to debug. The New Model shifts from copying to optimizing by using PPO under an explicit safety reward, while also upgrading conditioning (natural language via DistilBERT) and trust (GRU explanations). The projected outcome is higher Driving Score and fewer collisions on Longest6, with a modest latency increase that remains real-time.